

3-mavzu. Python dasturlash tilida satrlar

Reja.

1. Python dasturlash tilida satrni o'zgaruvchiga biriktirish. Ko'p qatorli satr, Satr – bu massiv.
2. Satrga oid funksiyalar. Satrlarni tekshirish.
3. Satrlar formati. Maxsus belgilar.

1. Python dasturlash tilida satrni o'zgaruvchiga biriktirish. Ko'p qatorli satr. Satr – bu massiv.

Python dasturlash tilida satrlar bitta yoki ikki qo'sh tirnoq ichida yoziladi. 'Salom' bilan "Salom" ikkisi ham bir xil hisoblanadi. Satrni qora ekranga chiqarish uchun `print()` funksiyasidan foydalanishingiz mumkin.

```
print("Salom")  
print('Salom')
```

```
Salom  
Salom
```

Satr turiga ega o'zgaruvchi yaratish.

Bir qatorli satrni o'zgaruvchiga biriktirish uchun to'g'ridan to'g'ri python dasturlash tilida uchramaydigan kalit so'zlarni ishlatmagan holda yozib ketsangiz bo'ladi.

```
a = "Hello"  
print(A)
```

```
Hello
```

Ko'p qatorli satrni o'zgaruvchiga biriktirish.

Ko'p qatorli satrni biriktirish uchun 3 ta tirnoq tarkibida yozish imkoni mavjud. quyidagi misolga e'tibor bering.

```
a = """  
Assalomu alaykum bugun siz bilan Python dasturlashini boshlaymiz.  
Ustozinigiz yoki Yo'l boshchingiz.  
Master Sherkulov  
"""  
print(A)
```

```
Assalomu alaykum bugun siz bilan Python dasturlashini boshlaymiz.  
Ustozinigiz yoki Yo'l boshchingiz.  
Master Sherkulov
```

Satrlar massiv hisoblanadi.

Boshqa ko'plab dasturlash tillari singari Python dasturlash tilida satrlar ifodalashda baytlardan iborat hisoblanadi. Biroq Python dasturlash tilida belgilar bo'yicha ma'lumot turlari mavjud emas, bitta belgini uzunligini shunchaki 1 deb qabul qilasiz. [] kvadrat qavs ichida ma'lumotlar ifodalanadi.

```
a = "Hello, World!"  
print(a[1])
```

Satrni ma'lum bir qismini qirqib olish va uni biron bir o'zgaruvchiga yuklash uchun yoki bo'lmasam ekranga print qilish uchun kvadrat qavs ichiga

boshlanishi va tugashini yozamiz va ikkitali nuqta orqali bir biridan ajratiladi. **[boslanish: tugash]**

```
b = "Hello, World!"  
print(b[2:5])
```

llo

Teskari qirqish uchun `-` dan foydalanamiz. Bunda oxirgi simbol indeksi birinchi yoziladi.

```
b = "Hello, World!"  
print(b[-5:-2])
```

orl

Qator uzunligini olish uchun `len()` funksiyadan foydalaning .

```
a = "Hello, World!"  
print(len(A))
```

13

Satrn barcha elementlarini kichik qillish uchun `lower()` funksiyasidan foydalaniladi.

```
a = "Hello, World!"  
print(A.lower())
```

hello, world!

Satrn barcha elementlarini katta qillish uchun `upper()` funksiyasidan foydalaniladi.

```
a = "Hello, World!"  
print(A.upper())
```

HELLO, WORLD!

Satrlarda ishlatiladigan belgilar.

Maxsus belgilar: Pythonda boshqa tillardagi kabi quyidagi maxsus belgilar mavjud:

- tabulyatsiya belgisi;
- yangi satrga o‘tish belgisi;
- \’ – apostrof belgisi;
- \” – qo‘sh tirnoq belgisi.

Quyidagi misolda yuqoridagi barcha maxsus belgilarni qo‘llangan holat uchun dastur keltirilgan.

Copy

```
print("1-chi kurs\n\"O'MU\"\ntalabasi")
```

Konsol ekraniga quyidagicha natija chiqariladi:

1-chi kurs

"O'MU" talabasi

Satrlarni taqqoslash: Satrlarni taqqoslashda satrda ishtirok etayotgan belgilarning registriga alohida e‘tibor qaratish lozim. Har qanday raqam ixtiyoriy alfavit belgisidan shartli kichik hamda katta registrli alfavit belgilari kichik registrli avfavit belgilaridan shartli kichik sanaladi. Masalan:

Copy

```
str1 = "1a"
```

```
str2 = "ab"
str3 = "Aa"
print(str1 > str2) # False, chunki str1 ning birinchi
# belgisi raqam
print(str2 > str3) # True, chunki str2 ning birinchi
# belgisi kichik registrga ega
```

Yuqoridagi dasturda `"1a" > "ab"` sharti `False` qiymat qaytaradi. Chunki raqam alfavit belgisidan shartli kichik hisoblanadi. Shuni alohida ta'kidlash kerakki, ikki satr solishtirilganda ularning mos tarkibiy elementlari solishritiladi (`"1a" > "ab"` holatda, dastlab 1 va "a" tekshiriladi). Agarda solishtirish natijasi teng bo'lsa navbatdagi mos elementlari solishtiriladi. Jarayon birinchi teng bo'lmagan holat topilganda yoki satrlardan birining oxiriga yetib kelinganda tugatiladi. Agar satrlarning dastlabki barcha mos elementlari teng, faqat ularning uzunliklari farqli bo'lsa, u holda uzunligi katta satr shartli katta bo'ladi. Masalan: `"abcd" < "abcde"` sharti `True`

Bundan tashqari satrlar ustuda amal bajaradigan `upper()` va `lower()` funksiyalari mavjud bo'lib, satr tarkibidagi alfavit belgilarni mos ravishda kichik va katta registrilariga almashtirish uchun ishlatiladi. Masalan:

```
Copy
str1 = "Kitob"
str2 = "kitob"
print(str1 == str2) # False - chunki ularni birinchi
# harflari turli registrda
print(str1.lower() == str2.lower()) # True chunki ikkala
# satr ham kichik registrga keltirilgan
```

9.2. Satrga oid funksiyalar. Satrlarni tekshirish.

Massiv bir vaqtning o'zida bir nechta qiymatlarni o'z ichiga oladigan maxsus o'zgaruvchidir. masalan quyidagi misolga qaranE.

```
cars = ["Ford", "Volvo", "BMW"]
```

Massiv elementlarini o'qish

Massiv elementlarini o'qish jarayonida siz **indeks** ga murojaat qilishinigiz kifoya.

```
print(cars[0])
```

```
Ford
```

Eslatma: Massiv elementlari **0** dan indekslanadi.

Massive uzunligi

Massiv uzunligi `len()` ko'makchi funksiyasidan foydalanib topish mumkin.

```
x = len(cars)
```

```
3
```

Massiv elementlarini alohida o'qish

for in kommandasidan foydalanib massiv elementlarini ajratib alohida print qilishingiz mumkin.

```
for x in cars:
    print(x)
```

```
Ford
Volvo
BMW
```

Massivga element qo'shish

append() Massivga element qo'shish uchun usuldan foydalanishingiz mumkin. Quyidagi misol ga etiboringiz qaratinE.

```
cars.append("UzAvto ;(")
```

```
Ford
Volvo
BMW
UzAvto ;(
```

Massivga element olib tashlash.

pop() va **remove()** ko'makchi funksiyalaridan foydalanishingiz mumkin.

```
cars.pop(1)
cars.remove("UzAvto ;(")
```

9.3. Satrlar formati. Maxsus belgilar.

Satr Unicode kodirovkasidagi belgilar ketma-ketligini ifodalashda qo'llaniladi.

Satrlarning alohida belgilariga kvadrat qavs ichida indekslarini ko'rsatish orqali murojaat qilishimiz mumkin:

Copy

```
string = "hello world" c0 = string[0] # h print(c0)
```

```
c6 = string[6] # w
```

```
print(c6)
```

```
c11 = string[11] # xatolik IndexError: satr indeksi chegaradan chiqib ketdi
```

```
print(c11)
```

Indekslash noldan boshlanadi, shuning uchun satrning birinchi belgisi 0 indeksiga ega bo'ladi. Agar satrda mavjud bo'lmagan indeksga murojaat qilinsa, *IndexError* istisno xatoligi yuzaga keladi. Masalan, yuqoridagi holatda, satr 11 ta belgidan iborat bo'lib, uning belgilari 0 dan 10 gacha indekslarga ega bo'ladi

Satrn oxirdan boshlab uning belgilariga murojaat qilish uchun manifiy indekslashdan foydalanish mumkin. Ya'ni, satrdagi eng oxirgi belgini -1 index orqali, oxirdan bitta oldingi belgisini -2 indeks orqali va hakoza tartibda olish mumkin.

Copy

```
string = "hello world" c1 = string[-1] # d print(c1)
```

```
c5 = string[-5] # w
```

```
print(c5)
```

Satrlar o'zgarmaydigan (immutable) tur hisoblanadi, shuning uchun ham satrni o'zgartirishga harakat qilinsa xatolik yuzaga keladi, masalan:

Copy

```
ss = "hello world"
```

```
ss[1] = "R"
```

Bu erda *ss* satrning 2-chi belgisiga yangi qiymat berish uchun *ss[1] = "R"* tarzda amal yozilgan. Natijada xatolik ro'y beradi. Chunki satr tarkibidagi belgilarni uning indeksi orqali o'zgartirishga yo'l qo'yilmaydi.

Faqatgina satrga boshqa qiymat yozish orqali satrni to'liq o'zgartira olamiz.

Satr ostilarini olish

Agar zarur bo'lsa, satrdan nafaqat bitta belgini, balki satr ostisini ham olishimiz mumkin. Buni amalga oshirish uchun quyidagi sintaksisdan foydalanamiz:

- `string[: end]` – 0-indeksdan boshlab `end`gacha belgilar ketma-ketligini oladi;
- `string[start: end]` – `start` indeksdan `end` indeksigacha bo'lgan belgilarni oladi;
- `string[start: end: step]` – `step` qadam bilan `start` indeksdan boshlab `end` indeksigacha bo'lgan belgilar kerma-ketligini oladi.

Satr ostisini olish uchun barcha variantlardan foydalanamiz:

Copy

```
string = "hello world"
# 0 dan 5 gacha belgilar sub_string1 = string[:5]
print(sub_string1) # hello
# 2 dan 5 gacha belgilar sub_string2 = string[2:5]
print(sub_string2) # llo
# 2 dan 9 gacha bitta belgi tashlab
sub_string3 = string[2:9:2]
print(sub_string3) # lowr
ord va len funksiyalar
```

Agar satr Unicode belgilaridan tashkil topgan bo'lsa, u holda `ord()` funksiyasi yordamida belgining Unicode kodlashdagi sonli qiymatini olish mumkin:

Copy

```
print(ord("A")) # 65
Satr uzunligi (satrdagi belgilar soni)ni olish uchun len() funksiyasidan foydalanish mumkin:
```

Copy

```
string = "hello world"
length = len(string) print(length) # 11
```

Satrdan izlash

Satrdan `term` satr ostisini qidirish uchun `term in string` ifodasidan foydalanamiz. Agar satr ostisi topilsa ifoda `True` qiymat qaytaradi, aks holda `False` qiymat qaytaradi:

Copy

```
string = "hello world" exist = "hello" in string
print(exist) # True
exist = "sword" in string print(exist) # False
```

Satrlarni ajratish

`for` sikl operatori yordamida satrning barcha elementlarini ajratib olish mumkin:

Copy

```
string = "hello world" for char in string:
print(char)
```